RL-Based GPU Job Scheduler

Hrujul Mendhe (B24CM1077) Atharv Sharma (B24CM1013)

GitHub: gpu-sched-rl

Abstract

We use reinforcement learning to schedule jobs on a shared GPU cluster. Our simulator models job arrivals, GPU memory availability, estimated runtimes, and deadlines. We train a PPO agent to choose which waiting job to run and how to place it on the GPUs. We also use observation and reward normalization to help stabilize training. We compare PPO with five scheduling methods and study its performance under two job-arrival rates.

1 Problem and Simulator

A GPU cluster runs jobs that may need different numbers of GPUs and different amounts of memory. Since GPU memory is limited, a job can start only when enough GPUs have the required free memory. The scheduler must decide which waiting job to start and where to place it. These decisions affect job completion time, waiting time, and missed deadlines.

We use a simulator to model this problem. It contains 16 GPUs, each with 40 GB of memory, and each episode has 60 jobs. Jobs request 1, 2, 4, or 8 GPUs, with 8–40 GB of memory per selected GPU. The simulator assigns each job a runtime and deadline. The agent receives an estimated runtime, which may differ from the actual runtime.

We focus on a harder synthetic workload in which larger jobs are more common and runtime estimates are less reliable. This helps test whether the scheduler can handle different job sizes and limited GPU memory. All experiments are conducted in simulation.

2 Reinforcement Learning Formulation

We model scheduling as a Markov decision process (MDP). At each step, the agent observes the current cluster and queue, chooses an action, and receives a reward.

State. The state contains 116 numbers:

$$|S| = 4 + 16 + (8 \times 12) = 116.$$

Four values describe the overall cluster and queue, 16 values describe the free memory on each GPU, and 12 values describe each of the first eight waiting jobs. Job information includes its GPU and memory requirements, estimated runtime, waiting time, deadline information, and whether it can currently be scheduled. If fewer than eight jobs are waiting, the remaining entries are filled with zeros.

Actions. The agent has 17 possible actions. For each of the first eight waiting jobs, it can choose `PACK` or `SPREAD`. The final action is `WAIT`.

`PACK` places a job on GPUs where it leaves less unused memory, helping preserve larger free-memory spaces elsewhere. `SPREAD` prefers GPUs with more free memory. Actions for jobs that cannot fit are masked, preventing the agent from selecting them. Starting a job does not advance time; choosing `WAIT` advances the simulation by one tick.

Reward. Starting a job gives no reward; reward is paid when the simulator advances one tick (action WAIT). Let Q_t be the waiting queue, C_t the jobs that finish this tick, and S_t the queued jobs that have waited more than 30 ticks. For a finishing job j with arrival a_j, finish time f_j and deadline d_j, the turnaround is $F_j = f_j - a_j$. The reward is

$$r_t = \sum_{j \in C_t} c_j \; - \; p_t, \tag{1}$$

with a per-job completion term

$$c_j = w_c - w_f \frac{F_j}{R_{\max}} - w_l \max(0,\, f_j - d_j) - w_m \,[f_j > d_j] \tag{2}$$

and a per-tick penalty

$$p_t = w_w |Q_t| + w_s |S_t| + w_i \,[\text{idle fit}]\, \frac{M_{\text{free}}}{M}. \tag{3}$$

Here $[\cdot]$ equals 1 if the condition holds and 0 otherwise, $R_{\max} = 100$ is the maximum runtime, and M_{free}/M is the free-memory fraction. The condition "idle fit" holds when one of the first eight queued jobs could start now but has not. The weights are $w_c = 1$, $w_f = 0.5$, $w_l = 0.03$, $w_m = 2$, $w_w = 0.015$, $w_s = 0.01$, $w_i = 0.05$. Completions are rewarded, while slow, late or deadline-missing ones, queue buildup, starvation and idle resources are penalized.

3 Training and Evaluation

We train the agent using MaskablePPO, a PPO algorithm that supports masking invalid actions. The policy uses a neural network with two hidden layers of 256 units each. We also use `VecNormalize` to normalize observations and rewards during training. The reported `ppo_v9_hard` run uses three million training timesteps and eight parallel environments, with SJF imitation pretraining disabled. During evaluation, the trained policy chooses actions deterministically and continues to use action masks.

We compare PPO against five scheduling methods: FIFO (first in, first out), SJF (shortest job first), EDF (earliest deadline first), backfill, and best-fit. Each method is evaluated on the same hard workload using 100 episodes at each arrival rate. The seen-load setting has an arrival rate of 0.4 jobs per tick, while the unseen-load setting uses 0.7 jobs per tick.

We measure average job completion time (JCT), deadline-miss rate, maximum wait, GPU allocation utilization, and memory utilization. Lower completion time and miss rate generally indicate better scheduling.

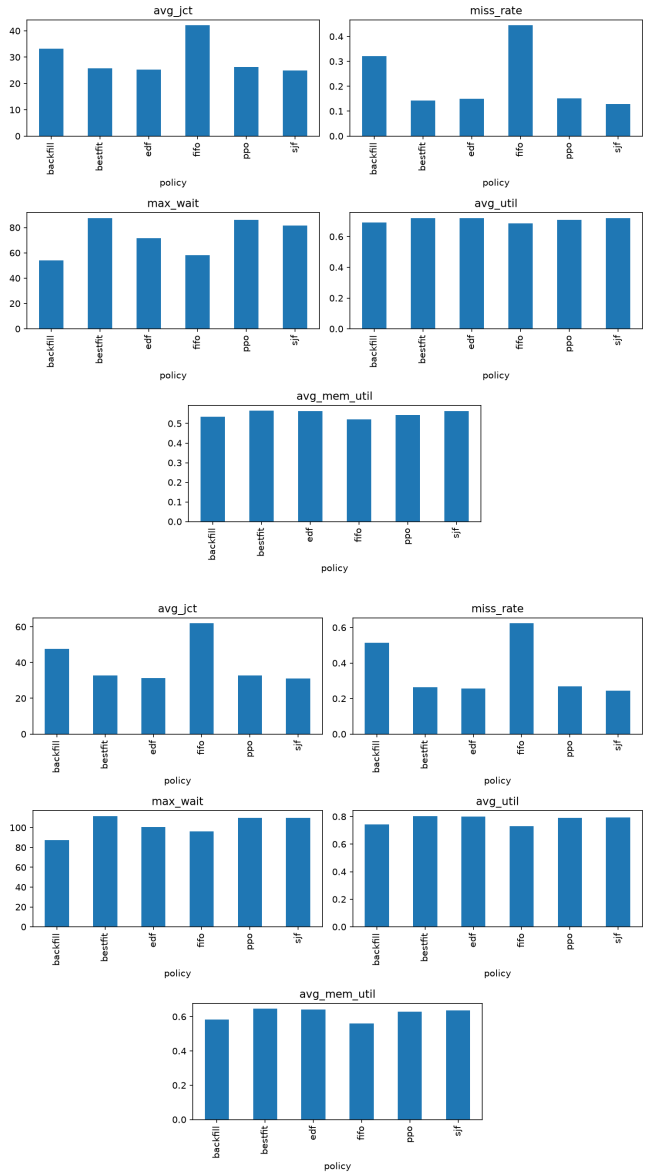


Figure 1: Comparison of the six scheduling methods on the hard workload at arrival rates 0.4 (top) and 0.7 jobs per tick (bottom).

4 Results and Discussion

Table 1 summarizes average completion time, deadline-miss rate, and GPU utilization. Figure 1 shows all five metrics for both arrival rates.

PPO demonstrates effective scheduling behavior across both workload settings, outperforming FIFO in average job completion time and deadline-miss rate. Its performance is also comparable with heuristic policies of SJF and EDF algorithms.

This comparison highlights the challenge of learning a scheduling policy that generalizes across competing objectives, such as reducing completion times, meeting deadlines, and maintaining resource utilization while also adjusting for noisy runtime estimates. While PPO does not surpass the best heuristics in the current evaluation, its learned policy provides a promising foundation for further improvements in memory-aware scheduling.

The graphs also show that the higher arrival rate makes scheduling harder: completion times and deadline-miss rates increase for every method. GPU and memory utilization are relatively similar across several methods. GPU allocation utilization measures how often GPUs have jobs assigned to them; it does not measure actual GPU computing activity. High utilization alone does not guarantee shorter waits or fewer missed deadlines.

Table 1: Mean results over 100 episodes per policy.

Load	Policy	JCT	Miss rate	GPU util.
0.4	FIFO	42.12	44.5%	68.5%
	SJF	24.89	12.9%	72.1%
	EDF	25.25	14.9%	72.0%
	Backfill	33.27	32.1%	69.1%
	Best-fit	25.80	14.3%	72.0%
	PPO	26.18	15.1%	70.9%
0.7	FIFO	61.96	62.4%	72.9%
	SJF	31.05	24.3%	79.5%
	EDF	31.25	25.5%	80.0%
	Backfill	47.59	51.4%	74.2%
	Best-fit	32.68	26.3%	80.4%
	PPO	32.78	26.8%	79.2%

5 Conclusion

We built a GPU scheduling simulator and trained a MaskablePPO agent to select jobs and placement strategies while respecting GPU memory limits. We used `VecNormalize` during training and compared PPO with five scheduling methods under two job-arrival rates. PPO performs substantially better than FIFO, achieves comparable performance with respect to SJF and EDF. Future improvements could focus on improving the reward design and testing the agent on more workloads and training runs.

References

[1] J. Schulman et al., "Proximal Policy Optimization Algorithms," arXiv:1707.06347, 2017.

[2] A. Raffin et al., "Stable-Baselines3: Reliable Reinforcement Learning Implementations," *Journal of Machine Learning Research*, vol. 22, no. 268, 2021.